34.5 投票系统

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~104k tokens，输出 ~4k tokens（本节）

在群组聊天中，Agent 有时需要发起投票来收集用户意见——比如“你更喜欢 A 方案还是 B 方案？”。OpenClaw 的投票系统（`src/polls.ts`）提供了一个平台无关的投票数据模型和校验层，具体的投票创建则委托给各聊天平台的 SDK 实现。

34.5.1 投票（Polls）实现（`src/polls.ts`）

数据模型

投票的核心数据结构非常简洁：

```
// src/polls.ts
export type PollInput = {
  question: string;        // 投票问题
  options: string[];       // 选项列表（至少 2 个）
  maxSelections?: number; // 最多可选几项（默认 1 = 单选）
  durationHours?: number; // 投票持续时间（小时）
};

export type NormalizedPollInput = {
  question: string;
  options: string[];
  maxSelections: number;  // 必填（已填充默认值）
  durationHours?: number;
};
```

`PollInput` 是外部输入的原始格式，`NormalizedPollInput` 是经过校验和默认值填充后的标准格式。

输入校验

`normalizePollInput` 实现了完整的输入校验链：

```
export function normalizePollInput(
  input: PollInput,
  options: { maxOptions?: number } = {},
): NormalizedPollInput {
  // 1. 问题不能为空
  const question = input.question.trim();
  if (!question) throw new Error("Poll question is required");

  // 2. 选项清洗：去除首尾空白 → 过滤空串 → 至少 2 个
  const cleaned = input.options.map(o => o.trim()).filter(Boolean);
  if (cleaned.length < 2) throw new Error("Poll requires at least 2 options");

  // 3. 选项上限检查（由平台决定，如 Discord 最多 10 个）
  if (options.maxOptions && cleaned.length > options.maxOptions) {
    throw new Error(`Poll supports at most ${options.maxOptions} options`);
  }

  // 4. maxSelections 校验：≥1 且 ≤ 选项数
  const maxSelections = typeof input.maxSelections === "number"
    ? Math.floor(input.maxSelections) : 1;
  if (maxSelections < 1) throw new Error("maxSelections must be at least 1");
  if (maxSelections > cleaned.length) throw new Error("maxSelections cannot exceed opti

  // 5. durationHours 校验：≥1（若提供）
  const durationHours = typeof input.durationHours === "number"
    ? Math.floor(input.durationHours) : undefined;
  if (durationHours !== undefined && durationHours < 1) {
    throw new Error("durationHours must be at least 1");
  }

  return { question, options: cleaned, maxSelections, durationHours };
}
```

持续时间归一化

`normalizePollDurationHours` 将投票持续时间限制在平台允许的范围内：

```
export function normalizePollDurationHours(
  value: number | undefined,
  options: { defaultHours: number; maxHours: number },
): number {
  const base = typeof value === "number" && Number.isFinite(value)
    ? Math.floor(value)
    : options.defaultHours;
  return Math.min(Math.max(base, 1), options.maxHours);
  // 结果保证在 [1, maxHours] 范围内
}
```

不同平台的限制各不相同。以 Discord 为例，投票最长可持续 32 天（`32 * 24 = 768` 小时），最多 10 个选项。

平台集成

投票模块本身是纯校验层，不包含任何平台特定逻辑。各通道插件负责将 `NormalizedPollInput` 转换为平台原生的投票格式：

平台	投票能力	最大选项数	最长持续时间	对应文件
Discord	✓	10	32 天	`src/discord/send.shared.ts`
WhatsApp	✓	-	-	`src/whatsapp/`
MS Teams	✓	-	-	`src/msteams/`
Telegram	✗	-	-	-
Web	✓	-	-	`src/web/outbound.ts`

Discord 投票示例：

Discord 的投票通过 REST API 的 `poll` 字段创建。OpenClaw 先调用 `normalizePollInput` 校验，再调用 `normalizePollDurationHours` 约束持续时间，最后构造 `RESTAPIPoll` 对象：

```
// src/discord/send.shared.ts（概念简化）
const normalized = normalizePollInput(pollInput, { maxOptions: 10 });
const durationHours = normalizePollDurationHours(normalized.durationHours, {
  defaultHours: 24,
  maxHours: 32 * 24,
});
const poll: RESTAPIPoll = {
  question: { text: normalized.question },
  answers: normalized.options.map(text => ({ poll_media: { text } })),
  duration: durationHours,
  allow_multiselect: normalized.maxSelections > 1,
  layout_type: PollLayoutType.Default,
};
```

通道能力声明

投票是通道的可选能力。通道注册时在 `capabilities` 中声明是否支持 `polls`：

```
// src/channels/dock.ts（Discord 通道能力声明示例）
{
  polls: true,    // 支持投票
  // ...其他能力
}
```

Agent 工具层在执行投票动作前会检查能力开关：

```
// src/agents/tools/discord-actions-messaging.ts
if (!isActionEnabled("polls")) {
  throw new Error("Discord polls are disabled.");
}
```

架构设计理念

投票系统的设计体现了 OpenClaw 的一个核心理念：**平台无关的语义层 + 平台特定的适配层**。

```
Agent 工具层
  ↓ 生成 PollInput { question, options, maxSelections, durationHours }
  ↓
normalizePollInput()  ← 平台无关的校验
  ↓
NormalizedPollInput
  ↓
通道适配层  ← 各平台转换为原生格式
  ├─ Discord: RESTAPIPoll
  ├─ WhatsApp: WhatsApp Poll Message
  └─ Web: ActiveListener 推送
```

这样的分层让添加新平台的投票支持变得简单——只需在通道插件中实现 `NormalizedPollInput` 到原生格式的转换，核心校验逻辑无需重复。

本节小结

1. **投票数据模型**由 `PollInput` / `NormalizedPollInput` 定义，包含问题、选项列表、最大选择数、持续时间四个字段。
2. **输入校验**由 `normalizePollInput` 统一完成——清洗选项文本、验证最少 2 个选项、检查 `maxSelections` 范围、验证持续时间。
3. **持续时间归一化**使用 `normalizePollDurationHours` 将值限制在 `[1, maxHours]` 范围内，不同平台有不同的上限。
4. **平台集成**通过能力声明（`polls: true`）和适配层实现，各通道将标准化的投票输入转换为平台原生格式。
5. **架构理念**是"平台无关语义层 + 平台特定适配层"，新增平台投票支持只需实现转换，无需修改核心校验。